SECTION 1 OF 3

Introduction to Neural Networks & Python Foundations

From the meaning of a neural network to your first working perceptron

Neurons & Motivation · Python & NumPy · The Perceptron

COURSE MAP

Where This Section Fits

1

Intro, Python & the Perceptron

What a neural network is, Python/NumPy tooling, the single neuron

2

Multi-Layer Networks & Activations

MLP architecture, high-dimensional inputs, activation functions, classification

3

Learning: Gradient Descent & Regularization

Loss & cost, gradient descent, the chain rule, backpropagation, regularization

→

Later: CNNs, the Final Project & PyTorch

Convolutional networks for images, the semester project, and a PyTorch walkthrough

COURSE PIPELINE

Semester Timeline — Topic by Topic

Material is released one topic at a time, in this order, so each week builds on the last.

1. Simple neural network (the perceptron)
2. Multi-layer neural networks
3. Gradient descent
4. Regularization
5. Classification task (graded mini-project)
6. Activation functions, in depth
7. The chain rule & backpropagation
8. Final project: CNNs on image data (+ PyTorch walkthrough)

What You'll Be Able to Do

- Explain, in plain language, what a neural network is and why it learns from data instead of fixed rules
- Set up a Python environment with the packages a neural-network workflow depends on
- Read and write core Python — data types, collections, control flow, functions
- Use NumPy arrays to represent inputs, weights, and outputs efficiently
- Build, by hand and in code, a single artificial neuron (a perceptron) and explain its formula

PART A

What Is a Neural Network?

The idea, the motivation, and the vocabulary we'll use all semester

What Is a Neural Network?

- A neural network is a computing system loosely inspired by the brain: many simple connected units, called neurons
- It learns patterns directly from examples, instead of being programmed with hand-written rules
- Formally, the whole network is just a function: it takes an input and produces an output
- Training means searching for the version of that function that fits our data best

Image Classification

input

a photograph

classifier

the neural network

output

"cat"

The same idea works for other tasks: a label per pixel (segmentation), a translated sentence, a fraud score…

Biological Inspiration: Neuron → Node

Biological Neuron	What It Does	Artificial Equivalent
Dendrites	Receive signals from other neurons	`Input vector x = [x1, x2, …, xn]`
Synapse	Strengthens or weakens a connection	`Weight wi on each input`
Cell body (soma)	Accumulates the incoming signal	`Weighted sum + bias: z = Σ(wi·xi) + b`
Axon / firing threshold	Fires a spike once a threshold is passed	`Activation function a = f(z)`

Why Neural Networks?

Image Classification

One label for the whole image — "cat", "5", "tumor"

Semantic Segmentation

A label for every pixel — used in self-driving cars, medical scans

Sequence Tasks

Text, speech, and time series, where order carries meaning

Structured Data

Tabular records — fraud detection, risk scoring, recommendations

A Very Short History

43

1943

McCulloch–Pitts

First mathematical model of a simplified, binary threshold neuron

58

1958

Rosenblatt's Perceptron

The first trainable single-layer neuron for classification

86

1986

Backpropagation Popularized

Rumelhart, Hinton & McClelland train multi-layer networks with gradients

12

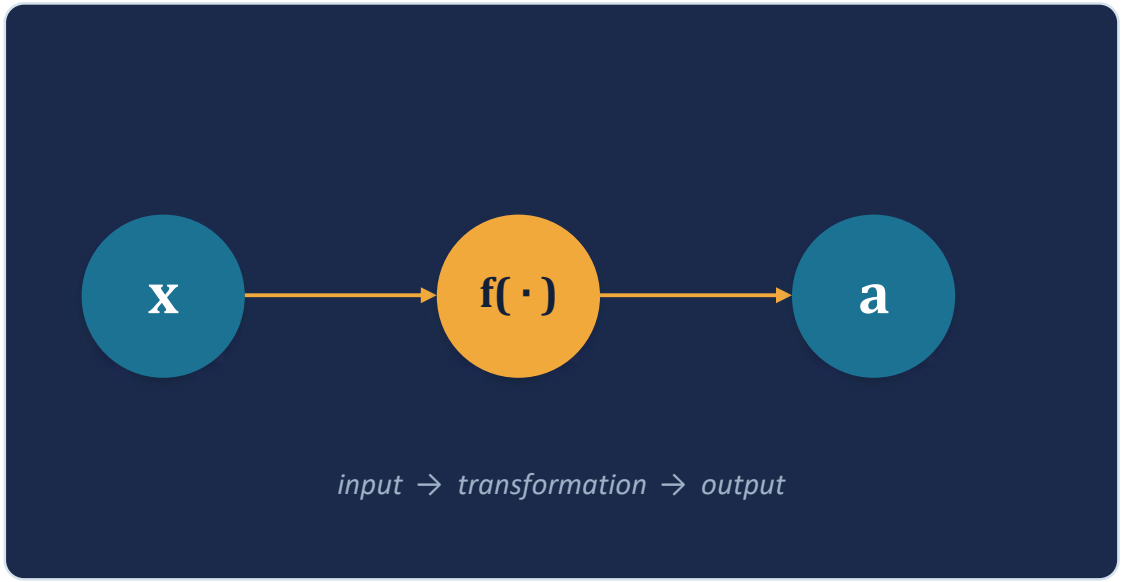
2012

The Deep Learning Era

GPUs + deep CNNs (AlexNet) revolutionize computer vision

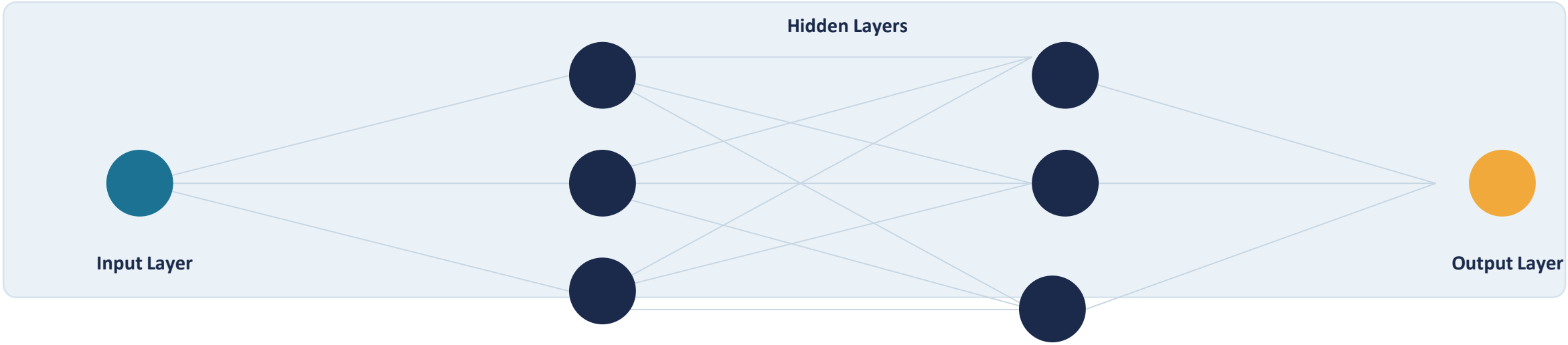
A Neuron Is Just a Function

- **Input — a set of numbers, x**
- *Transformation — a fixed rule that combines and reshapes those numbers*
- **Output — a single number, a (the neuron's activation)**



The Whole Network Is Also a Function

- Connect many neurons together and you still just have one big function
- The input layer takes in the data (an image, a row of a spreadsheet…)
- Hidden layers do the internal computation — most of the network's "work"
- The output layer produces the final result: a label, a score, a prediction



A Quick Preview of NN Types

We'll build these up gradually this semester — today's building block is the perceptron.

1

Perceptron

One neuron, linear decisions — Section 1

2

Multi-Layer Perceptron

Stacked layers, non-linear decisions — Section 2

3

Convolutional (CNN)

Spatial filters for images — final project

4

Recurrent / Transformer

Sequences: text, speech, time series — beyond this course

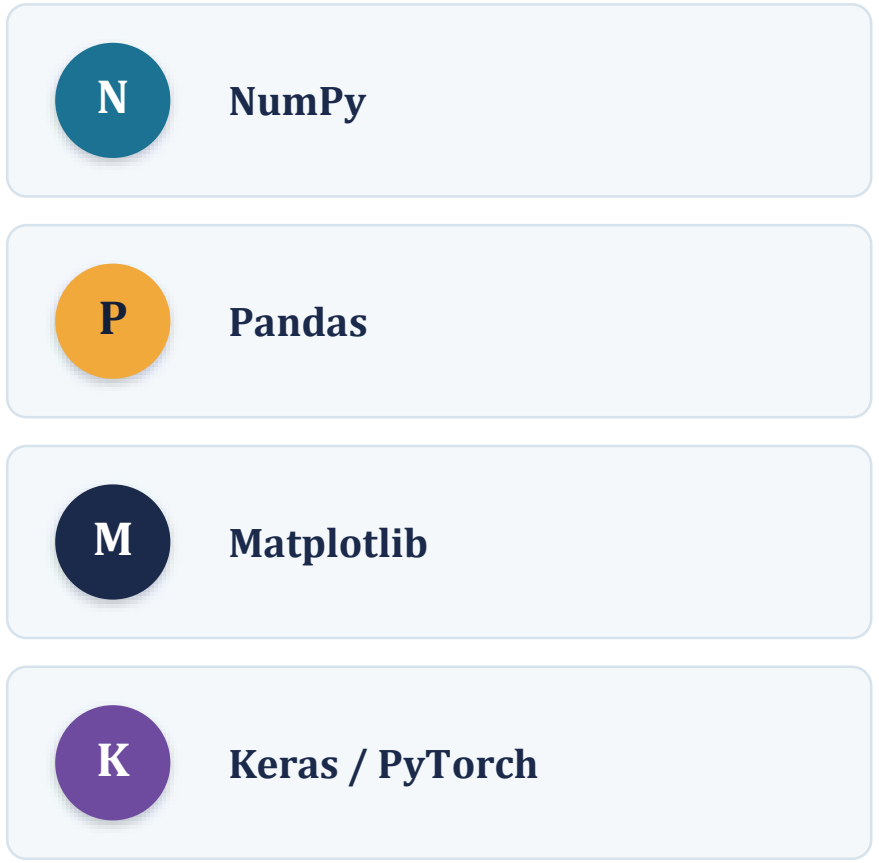
PART B

Python Foundations

The language and packages we'll use to build every network this semester

Why Python for Neural Networks

- Readable syntax — lets us focus on the neural-network ideas, not language mechanics
- A mature scientific stack: NumPy, Pandas, Matplotlib, SciPy
- Deep learning frameworks (Keras/TensorFlow, PyTorch) are Python-first
- The default language for this course's examples, notebooks, and the final project



Setting Up Your Environment

- Install Python 3.9+ and use a virtual environment per project
- Install the core stack with a single pip command
- Verify the install before writing any network code
- Notebooks (Jupyter) are recommended for experimenting one cell at a time

```
# create & activate a virtual environment
python -m venv nn-env
source nn-env/bin/activate
# (Linux/Mac)

# install the core packages
pip install numpy pandas matplotlib jupyter

# quick sanity check
python -c "import numpy;
print(numpy.__version__)"
```

Python Data Types

- int / float — numbers used for weights, inputs, and predictions
- str — labels and class names (e.g. "cat", "dog")
- bool — flags such as is_training
- Python infers each variable's type automatically

```
learning_rate = 0.01
# float
epochs = 50
# int
model_name = "Perceptron1"
# str
is_training = True
# bool

print(type(learning_rate))
# -> <class 'float'>
```

Lists, Tuples & Dictionaries

- list — ordered, changeable; store training samples or layer sizes
- tuple — ordered, fixed; a natural fit for a shape like (28, 28, 1)
- dict — key/value pairs; a natural fit for hyperparameters

```
layer_sizes = [3, 4, 1]
# list
input_shape = (28, 28, 1)
# tuple

config = {
    "learning_rate": 0.01,
    "epochs": 50,
}
print(config["epochs"])
# -> 50
```

NumPy: Arrays, the Building Block

- NumPy's ndarray stores numbers efficiently and computes on them fast
- Every input, weight, and output in this course is a NumPy array
- Arrays can be 1-D (a vector), 2-D (a matrix), or higher-dimensional

```python
import numpy as np

x = np.array([2.0, 3.0, -1.0])
# vector
w = np.array([0.5, -0.2, 0.1])
# weights

print(x.shape, x.dtype)
# -> (3,) float64
```

NumPy: Operations & Vectorization

- NumPy applies math to whole arrays at once — no manual loops
- This vectorization is what makes training on large datasets fast
- np.dot / the @ operator compute the weighted sum in one call

```
x = np.array([2.0, 3.0, -1.0])
w = np.array([0.5, -0.2, 0.1])
b = 1.0

z = np.dot(w, x) + b
# weighted sum
print(z)
# -> 1.3
```

Matplotlib: Visualizing Data

- Plots reveal patterns in data and check that training is working
- We'll use it constantly to plot loss curves and decision boundaries
- A rising loss curve is often the first sign something needs adjusting

```
import matplotlib.pyplot as plt

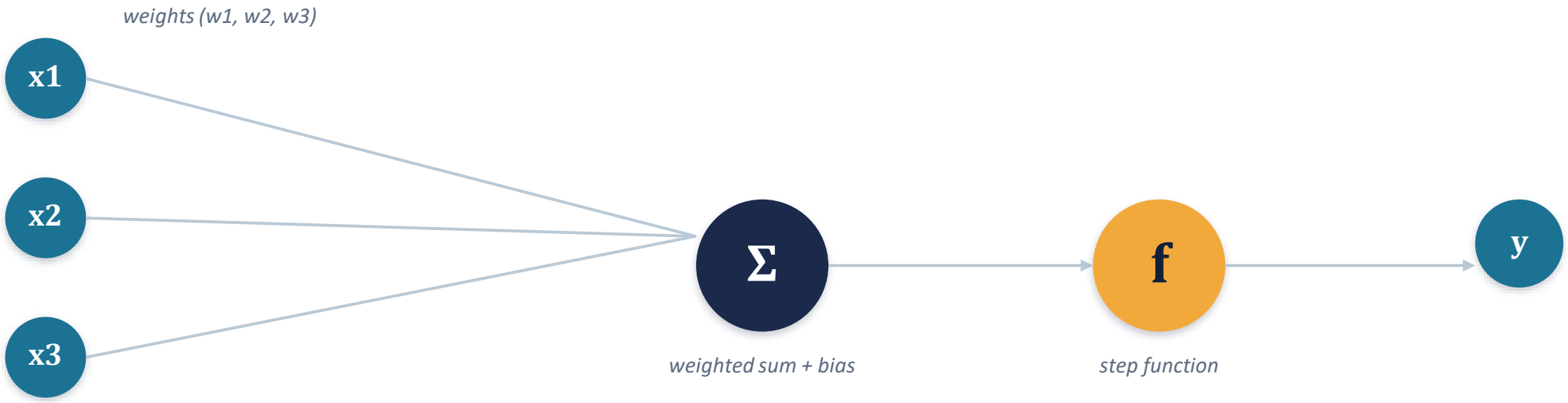
plt.plot(epochs, losses)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Progress")
plt.show()
```

PART C

The Simple Neural Network

One neuron, one decision: meet the perceptron

The Perceptron: One Neuron, One Decision




```
y = f( w1x1 + w2x2 + w3x3 + b )
```

Weighted Sum & Bias

- Weight (w) — how much each input matters; learned during training
- Bias (b) — a shiftable offset, so the neuron can fire even with small inputs
- A larger |weight| means stronger influence; the sign shows excitatory vs. inhibitory effect
- z = w1x1 + w2x2 + … + wnxn + b — this is just a dot product plus a constant

```
z = w · x + b   (NumPy: z = np.dot(w, x) + b)
```

The Activation Step

- The weighted sum z is a raw, unbounded number — the activation function turns it into a decision
- The classic perceptron uses a step function: fire (1) if z ≥ 0, otherwise stay silent (0)
- This single rule is enough to draw a straight-line decision boundary between two classes
- We'll meet smoother activations (sigmoid, ReLU, tanh) in Section 2 — they matter once we stack layers

```
f(z) = 1  if z ≥ 0  else  0
```

Linear Separability & Decision Boundaries

- A perceptron classifies by checking which side of a line (or hyperplane) a point falls on
- If a straight line can separate the two classes, the perceptron is guaranteed to find one
- This is called linear separability — it's the perceptron's core strength and its core limit
- Problems like XOR are not linearly separable — solving them needs more than one neuron (Section 2)

Worked Example: Perceptron by Hand

- Inputs: x1 = 2.0, x2 = 3.0
- Weights: w1 = 0.5, w2 = -0.2
- Bias: b = 1.0

Step 1 — weighted sum

$$z = (2.0 \times 0.5) + (3.0 \times -0.2) + (-1.0 \times 0.1) + 1.0$$
$$z = 1.0 - 0.6 - 0.1 + 1.0 = 1.3$$

Step 2 — activation (step function)

$$z = 1.3 \geq 0 \rightarrow \text{output } y = 1$$

The neuron fires — this input is classified as class 1.

Code: A Perceptron in NumPy

```python
import numpy as np

def step(z):
    return 1 if z >= 0 else 0

def perceptron(x, w, b):
    z = np.dot(w, x) + b
    return step(z)

x = np.array([2.0, 3.0])
w = np.array([0.5, -0.2])
b = 1.0

print(perceptron(x, w, b))
# -> 1
```

What's happening

- np.dot(w, x) computes the weighted sum
- Adding b gives the raw score z
- step(z) turns z into a 0/1 decision
- This function is the entire perceptron

SECTION 1 RECAP

From Meaning to Your First Neuron

We covered

- What a neural network is, and why
- Python & NumPy essentials
- The perceptron — a single trainable neuron

Key formula

- $z = w \cdot x + b$
- $y = f(z)$, f = step function
- Linear separability = the perceptron's limit

Up next: Section 2

- Stacking neurons into layers
- Images as high-dimensional vectors
- Sigmoid, ReLU, tanh & softmax